CompRep: A Dataset For Computational Reproducibility

Lázaro Costa
lazaro@fe.up.pt
University of Porto &
INESC TEC, Portugal

Susana Barbosa
susana.a.barbosa@inesctec.pt
INESC TEC, Portugal

Jácome Cunha
jacome@fe.up.pt
University of Porto &
HASLab/INESC TEC, Portugal

ABSTRACT

Reproducibility in computational science is increasingly dependent on the ability to faithfully re-execute experiments involving code, data, and software environments. However, assessing the effectiveness of reproducibility tools is difficult due to the lack of standardized benchmarks.

To address this, we collected 38 computational experiments from diverse scientific domains and attempted to reproduce each using 8 different reproducibility tools. From this initial pool, we identified 18 experiments that could be successfully reproduced using at least one tool. These experiments form our curated benchmark dataset, which we release along with reproducibility packages to support ongoing evaluation efforts.

This article introduces the curated dataset, incorporating details about software dependencies, execution steps, and configurations necessary for accurate reproduction. The dataset is structured to reflect diverse computational requirements and methodologies, ranging from simple scripts to complex, multi-language workflows, ensuring it presents the wide range of challenges researchers face in reproducing computational studies. It provides a universal benchmark by establishing a standardized dataset for objectively evaluating and comparing the effectiveness of reproducibility tools.

Each experiment included in the dataset is carefully documented to ensure ease of use. We added clear instructions following a standard, so each experiment has the same kind of instructions, making it easier for researchers to run each of them with their own reproducibility tool. The utility of the dataset is demonstrated through extensive evaluations using multiple reproducibility tools.

CCS CONCEPTS

• **General and reference** → **Evaluation**; *Empirical studies.*

KEYWORDS

Reproducibility, Open Science, Empirical Evaluation, Dataset

ACM Reference Format:
Lázaro Costa, Susana Barbosa, and Jácome Cunha. 2025. CompRep: A Dataset For Computational Reproducibility. In *ACM Conference on Reproducibility and Replicability (ACM REP '25), July 29–31, 2025, Vancouver, Canada.* ACM, Rennes, France, 11 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

ACM REP '25, July 29–31, 2025, Vancouver, Canada

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM.
https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 INTRODUCTION

Reproducibility is a fundamental aspect of the scientific method, essential for validating results and building on existing knowledge [35, 53, 71]. However, achieving scientific reproducibility is quite challenging, as reported in numerous studies [34, 61].

Science often relies on some kind of computational work, thus making reproducibility also a computational problem. We term this computational work underlying science *computational experiments*, which we define as collections of files written in any programming language (PL), together with data and other required files or dependencies, necessitating specific configurations within a computational environment to ensure consistent execution. Despite its importance, achieving reproducibility of computational experiments presents significant challenges. The variability of computational environments (including PLs and their dependencies), rapid software evolution (for instance, new versions of compilers, libraries, or APIs), and inadequate documentation of procedures (from dependencies' management to code execution), leads to difficulties in replicating results. These challenges undermine the credibility of scientific findings and hinders scientific progress [10, 23, 42, 55].

While multiple tools have been developed to address reproducibility challenges, their effectiveness is difficult to compare due to the absence of common benchmarks [26, 73, 79]. Researchers often struggle to assess these tools consistently, making it challenging to determine their strengths, limitations, and real-world applicability [18].

Given these challenges, there is a pressing need for a curated dataset that systematically documents and organizes computational experiments to support reproducibility research [75]. A well-structured dataset must encompass a broad spectrum of scientific fields, ensuring that it reflects the diverse computational requirements and methodologies used across disciplines [26, 73, 79]. Moreover, it should incorporate experiments of varying complexity, ranging from straightforward scripts to intricate, multi-language workflows involving databases and specialized software environments [60, 72].

In addition to covering a wide range of domains and complexities, the dataset should provide explicit details of software dependencies, execution steps, expected results, and any necessary configurations [20]. This level of documentation is essential for ensuring that experiments can be reliably reproduced in different environments without ambiguity or missing components [73]. Furthermore, the dataset should function as a universal benchmark, establishing a standardized set of experiments for objectively evaluating and comparing reproducibility tools [26, 79]. By providing a consistent reference point, researchers can systematically assess tool performance, identify limitations, and drive improvements in reproducibility practices [60, 72].

To address this gap, we propose a curated dataset that reflects the complexity and diversity of modern computational research, covering diverse domains, aiming to capture the wide range of challenges researchers face. Each experiment introduces common reproducibility challenges, such as managing software dependencies, handling diverse data formats, and navigating heterogeneous computing environments. By collecting experiments from diverse domains and carefully documenting their characteristics, this dataset provides a set of computational experiments for addressing reproducibility challenges and assessing reproducibility tools' capabilities. We provide more details about the source of the experiments in Section 3 and fully characterize them in Section 4. Importantly, our goal is to reproduce the experimental results originally reported by the creators of these experiments. This involves verifying whether the same outputs and findings can be independently obtained using the provided code, data, and documented procedures.

To construct this benchmark, we collected 38 computational experiments from diverse scientific domains and evaluated them using 8 different reproducibility tools. Through this evaluation, we found that 18 of these experiments (47%) could be successfully executed using at least one tool. This highlights the inherent challenges and limitations in current reproducibility practices, including inconsistent or incomplete documentation and environment setup. The successfully reproduced experiments form our curated dataset, which we make publicly available along with reproducibility packages. We present the evaluation and the resulting curated dataset in Section 5.

In Section 2, we discuss related approaches and prior studies that have addressed similar challenges. Section 6 analyzes potential limitations in experiment selection, dataset representativeness, and execution conditions. Finally, we discuss our results in Section 7 and present the main conclusions in Section 8.

2 RELATED WORK

Datasets specifically designed for testing and evaluating tools across different fields have played a pivotal role in benchmarking and advancing research. For example, the ImageNet Large Scale Visual Recognition Challenge (ILSVRC) [68] has been crucial in progressing computer vision by providing a large-scale dataset for benchmarking image recognition algorithms. Similarly, the Penn Treebank [51] has supported advancements in natural language processing by offering annotated corpora for evaluating parsing and tagging tools. In the realm of image processing and machine learning, the MNIST Database [22] of handwritten digits has been fundamental. It offers 70,000 labeled images of handwritten digits, widely used for benchmarking image recognition algorithms. Similarly, the COCO (Common Objects in Context) dataset [49] has been essential for advancing object recognition technologies by providing complex scene images with detailed object annotations and enhancing tasks like object detection and image captioning. In the field of natural language processing and social behavior analysis, the Yelp Dataset Challenge [7] offers a plethora of user-generated content, business attributes, and social interaction data, enabling a wide range of studies from sentiment analysis to economic forecasting. Additionally, the Microsoft Malware Classification Challenge (BIG 2015) [66] serves the security domain by providing a substantial set of malware samples for developing classification and clustering algorithms.

Although datasets are crucial for many aspects of science, and in particular for evaluations, in the context of reproducibility, there is no such contribution. Related approaches include earlier efforts, such as SciInc [83] and SciUnit [78], focused on proposing reproducibility approaches. Although they include a few computational experiments used to evaluate their own contribution, their goal was not to provide a benchmark that could be widely used. On the other hand, our work significantly extends the evaluation landscape by introducing a comprehensive dataset of 18 experiments designed to challenge and assess reproducibility tools more robustly.

Further advancements were made when researchers evaluated and compared eight key reproducibility tools using a constrained dataset of three experiments Costa et al. [18]. While insightful, this approach was limited by its narrow experimental range, which might not fully capture broader applicability and deeper reproducibility challenges. Our paper expands the scope considerably by curating a dataset that spans multiple scientific disciplines, thereby offering a more extensive platform for tool evaluation.

3 COLLECTING EXPERIMENTS

In this section, we define the scientific domains covered, and we describe how we collected the experiments. In this study, an "experiment" is defined as a collection of files written in any PL, encompassing data, configurations, database information, and other necessary files. These components are essential for executing the experiment consistently within a specific computational environment to reproduce the intended results. It is important to note that we exclusively considered experiments that do not depend on specific hardware features such as GPU, or CPU, thus excluding those that require high-performance computing resources. This study included any experiments where it is only necessary to install the required environment, and the results are independent of the hardware used. However, it is important to note that the outcome of these experiments, especially in artificial intelligence (AI) focused on model training, may not be the same in every execution.

3.1 Choosing the Experiments

To develop a robust and diverse dataset, we select a collection of experiments spanning multiple research fields.

The first is software engineering, represented by experiments published in the *IEEE/ACM International Conference on Software Engineering* (ICSE), a leading venue in the field. The second is databases, with experiments sourced from the *International Conference on Very Large Databases* (VLDB), another highly regarded event in its domain. In addition, we expanded our scope to include experiments from the field of *AI*; for that, we turned to the Zenodo repository, a well-known platform for open science [30]. Moreover, we expanded our scope to include foundational research in software engineering. These studies were sourced from the *European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (ESEC/FSE), a well-regarded venue that frequently features innovative research in software engineering and user-centric methodologies.

Additionally, we collected experiments related to climate change; for that, we explored journals such as *Climate Change*[1] and *Nature Climate Change*[2]. However, the articles do not provide sufficient details about the computational setup, making it impossible to reproduce the experiments. In this way, we turned to the Zenodo repository to collect experiments in this field.

Furthermore, we considered experiments within the medical and economics domain. The experiments for both these fields were sourced using Zenodo as our primary platform.

To further enrich the dataset, we explored works addressing reproducibility tools, expecting these would contain some experiments used to illustrate the use of the reproducibility tools. We considered 19 articles reporting reproducibility tools, namely Binder [11], Comprehensive Archiver for Reproducible Execution (CARE) [43], Code, Data, Environment (CDE) [36], Software Provenance in CDE (CDE-SP) [59], Code Ocean [15], Encapsulator [56], FLINC [3], PARROT [77], Provenance-To-Use (PTU) [58], Prune [41], RenkuLab [67], ReproZip [70], reprozip-jupyter [64], ResearchCompendia [74], SciInc [83], SciUnit [78], Science Object Linking and Embedding (SOLE) [50], Umbrella [52], and Whole Tale [9]. This ensures the reuse of the largest number of previously used experiments to evaluate existing reproducibility tools.

3.2 Getting the Experiments

We began by collecting the experiments published at the three prominent scientific conferences that were chosen. Thus, we randomly selected five publications from each of the following conferences: VLDB 2021[3], ICSE 2022[4], and ESEC/FSE 2023[5]. For the last one, we ensure that the selected works have user study experiences.

For the Zenodo repository, we performed four targeted searches using the keywords "Medical", "Artificial Intelligence", "Climate Change", and "Economics" for the finding entries related to the respective field. For each search, we filtered results to include only software repositories, focusing on the first 100 entries. From these, we randomly selected five repositories per query to ensure a balanced representation of experiments across the specified domains.

Among the articles reporting reproducibility tools, only two – *SciInc* [83] and *SciUnit* [78] – contained available and complete experiments. From these, we extracted three specific experiments: *Chicago Food Inspections Evaluation* [16], *Variable Infiltration Capacity* [29] and *Incremental Query Execution* [37].

3.3 Summary

After this process, we were able to collect 38 experiments. Thus, we have 13 experiments from software engineering, 5 from climate change, 5 from medical research, 5 from AI, 5 from economics, and 5 from user studies.

[1]https://www.springer.com/journal/10584/
[2]https://www.nature.com/nclimate/
[3]http://vldb.org/pvldb/volumes/15/
[4]https://ieeexplore.ieee.org/xpl/conhome/9793835/proceeding?isnumber=9793541
[5]https://dl.acm.org/doi/proceedings/10.1145/3611643

4 CHARACTERIZATION OF COLLECTED EXPERIMENTS

For each of the 38 experiments collected, we categorized each one in Table 1. In the next section, we present the subset of these experiments that form our curated dataset — selected based on successful reproducibility — which is publicly available to support community use and future extensions [**?**].

We present (in this order) a link to the original repository (clickable in the PDF version of the work), the original publication, the PLs used and the database engine used, the experiment size, the number of files of each experiment, whether the dependencies are described (✓ indicates that there is a file with all the details necessary to build the computational environment of the experiment, ✗ means that there is not a description, and ✓✗ indicates that the dependencies and PLs are listed but without specifying the versions used), and finally the indication of how to execute the experiment (✓ indicates that detailed execution steps are provided, ✗ indicates insufficient information, "Package" is used for repositories that are libraries, packages, or similar artifacts that do not produce a specific result, and "VS Code Plugin" is used to mark repositories that are a plugin for VS Code, which are not supposed to produce any particular result and are thus not considered for reproducibility). "Not available" is used to mark a repository that is not available, and "Questionnaire" is used to mark a repository that only has the content of the questionnaire and no code to execute. Both are not supposed to produce any particular result and are thus not considered for reproducibility therefore, they are excluded from the reproducibility considerations of this study, leaving 36 experiments. Additionally, we report whether each experiment had previously received any ACM artifact evaluation badges, based on the ACM's *Artifact Review and Badging (v1.1)*[6] guidelines, applied as follows:

- ***Artifacts Evaluated - Functional*** – The artifact has been reviewed and verified to be documented, complete, and functional.
- ***Artifacts Evaluated - Reusable*** – The artifact provides significant functionality and is well-documented for reuse by other researchers.
- ***Artifacts Available*** – The artifact is stored in a publicly accessible repository with a unique identifier, ensuring long-term availability.
- ***Results Validated - Reproduced*** – This artifact indicates that the main results of the original study have been successfully obtained by an independent person or team using, at least in part, the artifacts provided by the original authors. In our evaluation, we focused on this category by executing the complete source code from each repository to verify reproducibility. We did not consider the *Results Validated - Replicated* badge, as replication requires an independent reimplementation using different methods or conditions.

The collected experiments span a wide range of PLs, reflecting the diversity of computational approaches utilized in scientific research. Notably, Python is the most frequently used PL, present in

[6]ACM. Artifact Review and Badging (v1.1), accessed on March 30, 2025, https://www.acm.org/publications/policies/artifact-review-and-badging-current

Table 1: Characterization of the Collected Experiments

Repo. Link	Publication	PL/Database	Experiment Size	# Files	Dependencies Description	How to execute	ACM badges
link	Adnan et al. [1]	Unix Shell, Python	4.6 GB	57	✓	✓	
link	Bai and Zhao [8]	Python	241.5 MB	50	×	✓	
link	Chauhan et al. [13]	C++	125.6 MB	71	×	✓	
link	Sun et al. [76]	Python /PostgreSQL	364.9 MB	3636	×	✓	
link	Zhou et al. [85]	C++	86.2 MB	11	✓	✓	
link	Chen et al. [14]	Python	2.3 MB	85	✓	✓	–
link	Kukucka et al. [46]	Java, Python, Unix Shell	153.4 MB	791	✓	✓	
link	Nguyen and Grunske [54]	Python, Java	2.9 MB	758	✓	✓	–
link	Xie et al. [81]	Python	10.4 MB	66	✓	✓	–
Not available	Zhang et al. [84]		Not available				–
link	Edwards [27]	R	488.7 KB	69	✓	Package	–
link	Hemes [39]	R	71.4 KB	3	✓	×	–
link	Jehn [44]	Python	2.7 MB	64	×	×	–
link	Lin et al. [48]	Python	128.8 MB	208	×	✓	–
link	Qasmi and Ribes [63]	R	70.3 KB	31	✓	Package	–
link	Fries [31]	Python	21.3 MB	217	×	✓	–
link	Han [38]	Python	139.1 KB	33	✓	✓	–
link	Hoyt [40]	Python	261.8 KB	67	✓	Package	–
link	Kailas et al. [45]	Python, Unix Shell	1 MB	167	✓	✓	–
link	Yang et al. [82]	Python	38.7 MB	37	✓	✓	–
link	Prezja [62]	Python	3.8 MB	91	✓	Package	–
link	Agresta et al. [2]	Python	352.8 KB	6	✓	×	–
link	Erickson [28]	Python, Unix Shell, C++	5.5 MB	459	✓	Package	–
link	Coupette et al. [19]	Python, Jupyter Notebook	114.4 MB	93	✓	×	–
link	Doe [24]	Python	1.9 MB	25	✓	×	–
link	Collins et al. [16]	R	147.9 MB	101	✓	✓	–
link	Essawy [29]	Python, Unix Shell	254 KB	60	×	×	–
link	Hai [37]	Python /SQLite	68.6 KB	26	×	×	–
link	Dritsa et al. [25]	Jupyter Notebook	1.1 GB	59988	✓	✓	–
link	Cai and Smeeding [12]	R	4.6 MB	31	×	×	–
link	Arcidiacono et al. [6]	R	182.9 KB	15	✓	✓	–
link	Spinellis et al. [69]	R, Python, Unix Shell	327.9 KB	15	✓	×	–
link	Almås et al. [5]	R	183.7 MB	4062	✓	✓	–
link	Davis et al. [21]	TypeScript, JavaScript	1.1 MB	81	✓	VS Code Plugin	
link	Ganji et al. [33]	TypeScript, JavaScript, Unix Shell	98.7 MB	14733	✓	VS Code Plugin	
link	Ritonummi et al. [65]		Questionnaire				–
link	Ahmed et al. [4]	Python, Unix Shell	4 MB	70	✓	✓	
link	Fronchetti et al. [32]	Python	134.4 MB	6409	✓	✓	–

67% (24 of 36) of the experiments, underscoring its dominance in scientific computing and data analysis. Unix Shell and R are the second most common, used in 22% (8 of 36) of the experiments, highlighting their importance in automating workflows and statistical analysis. C++ is utilized in 8% (3 of 36) of the experiments. Finally, TypeScript, JavaScript, Java, and Jupyter Notebook each represent 6% (2 of 36).

The size of the collected experiments varies significantly, showcasing its heterogeneity. The smallest experiment has 68.6 KB, while the largest exceeds 4.6 GB, illustrating a broad spectrum of computational complexities. Similarly, the number of files per experiment ranges from as few as three files to as many as 59,988 files, further demonstrating the variation in experiment structure and scope.

In terms of reproducibility, these experiments reveal notable gaps. The data, detailed in Table 1, shows that while 67% (24 of 36) of the experiments include all the details necessary to build the computational environment of the experiment, a significant proportion still lacks this critical information. Furthermore, only 50% (18 of 36) of the experiments provide detailed guidance on how to execute them, which is essential for ensuring replicability in different computational environments. These gaps highlight the challenges researchers face when attempting to reproduce computational experiments from existing publications [57].

We also examined whether the collected experiments had been awarded any ACM artifact evaluation badges. Of the 38 experiments, 9 (24%) had the Artifacts Available badge. Among these, only 2 experiments received the more stringent Artifacts Evaluated – Reusable badge. Notably, the majority of experiments—29 out of 38—had no badge at all.

Overall, the collected experiments present a diverse and heterogeneous collection of experiments in terms of PL, size, and file count. However, it also exposes significant shortcomings in the documentation of dependencies and execution instructions, which are vital for advancing reproducibility in scientific research.

In the following section, we introduce the curated dataset, which showcases experiments designed to function as a universal benchmark.

5 EVALUATING THE COLLECTED EXPERIMENTS

This section presents the results of our efforts to reproduce the collected experiments described in Table 1 using eight reproducibility tools: Binder [11], Code Ocean [15], FLINC [3], PTU [58], RenkuLab [67], ReproZip [70], SciUnit [78], and Whole Tale [9], which were previously identified in a comprehensive study [18]. To ensure long-term accessibility and reproducibility, all curated experiments, metadata, and reproducibility packages are publicly archived on Zenodo [**? ?**].

Table 2 details each tool's location (either a GitHub repository or a dedicated webpage) and whether the tool was accessed online or installed locally. All tools were accessed in October 2024. In the initial phase of executing the experiments, we were able to use the Whole Tale platform. However, when we attempted to add more collected experiments, the platform became unavailable.

Experiments executed locally were run on a personal computer equipped with a 6-core CPU and 16 GB of RAM, operating under Ubuntu 20.04.

Table 2: Summary of the Tools Used for Evaluating the Dataset

Tool	Location	Access Mode
Binder	https://mybinder.org/	Online
Code Ocean	https://codeocean.com/	Online
FLINC	https://github.com/depaul-dice/Flinc	Installed Locally
PTU	https://github.com/depaul-dice/provenance-to-use	Installed Locally
RenkuLab	https://renkulab.io/	Online, Version 2.0
ReproZip	https://www.reprozip.org/	Installed Locally
SciUnit	https://github.com/scidash/sciunit	Installed Locally
Whole Tale	https://wholetale.org/	Online

5.1 Running the Experiments

For each experiment that we collected, we manually installed the necessary dependencies and attempted to reproduce the results using each tool. We verified the outcomes by comparing the generated outputs against the results reported in the original publications. We standardized the preparation and documentation process for each experiment, and we added the file *READ-FOR-REPRODUCIBILITY.md* to each experiment. Although we strived to follow a consistent structure for describing the execution environment, the execution process itself, and how the information is presented, there is currently no universally accepted template for this type of documentation. As discussed in [47], several metadata formats and documentation standards exist across domains, but no consensus has emerged on a single best practice for reproducibility. Therefore, we selected and organized the information we considered most relevant to support consistent reproduction.

The outcomes are summarized in Table 3, which includes the reproducibility status and the generated package size (in MB, using the Zip format) when applicable. For failed executions, specific reasons are annotated, such as unsupported PL, compilers, databases, or tool limitations. To support transparency and future reuse, we created a reproducibility package for each experiment-tool pair where execution was successful. These packages include the source code, data, execution scripts, and are publicly available [**?**]. The results presented in Table 3 and Table 4 were compiled manually in structured spreadsheets based on our observations and verification of the original outputs. This format enables other researchers to easily extend the evaluation by adding new tools as additional columns and recording their outcomes in a consistent manner. While the

execution and verification steps are performed manually, the organization of the data is designed to ensure reproducibility and to support ongoing contributions from the community.

The reproducibility status is labeled as follows: "✓" – the experiment was successfully reproduced, and "✗" – the experiment could not be reproduced. We use specific labels to classify repository availability and reproducibility. "No Data" signifies that the necessary data for reproduction is missing from the repository. "Missing Files" is used when specific files are absent, leading to execution errors. "Partial" applies to repositories that are only partially reproducible, usually because some required data is restricted by licensing. For experiments where the platform became unavailable, we marked those experiments with a "-".

We could not run the experiment from Nguyen and Grunske [54] in any framework, even in the ones that fully support the Java environment (Code Ocean, PTU, ReproZip, and sciunit), because we could not find a Java class required by the experiment. For the experiments conducted by Kailas et al. [45] and Ahmed et al. [4], the required version of Python was not documented. We attempted execution using Python versions 3.5, 3.7, 3.8, and 3.9; however, all were found to be incompatible with several necessary packages, thus preventing successful experiment execution. Additionally, for the experiments conducted by Arcidiacono et al. [6] and Yang et al. [82], we encountered unresolved conflicts during the package installation process. For the experiments conducted by Doe [24], we encountered an unexpected input data format, which rendered execution impossible.

Notably, we were not able to run these experiments on our personal computers either. The problem may be related to a possible error in the source code or some failure in the configuration of the computational environment due to a lack of information. Hence, it is crucial to provide not only the source code of the experiment but also all required information for effective execution, including data, details about the computational environment used, and any other relevant execution details.

For the experiments by Cai and Smeeding [12], Spinellis et al. [69], and Almås et al. [5], we had access to only a portion of the complete set of data of the experiment. Although we contacted the authors to request full access, we were unable to obtain the necessary license. As a result, we could only partially reproduce the experiment.

From the initial selection of 38 experiments, we encountered several challenges that influenced the final curated dataset to be used to test computational reproducibility tools. Five of the experiments in the initial collection were identified as packages or libraries, which cannot be included in a reproducibility study as they do not execute or produce any specific results tied to the corresponding publications. Additionally, six experiments lacked access to either the source code or the required data, rendering them impossible to execute. Furthermore, two experiments were VS Code plugins, which do not produce expected outputs and are not supported by any of the evaluated platforms. One experiment was a questionnaire containing only the content of the questionnaire itself, which was thus also excluded. Finally, six experiments could not be successfully executed on any platform despite meeting other criteria and were therefore excluded from the curated dataset.

5.2 Curated Datasset

In Table 4, we define the curated dataset of experiments that only includes those that have been successfully reproduced by at least one of the eight reproducibility tools considered. This curated selection focuses exclusively on experiments that produce a tangible result, excluding cases such as R packages and plugins, which are not intended to generate specific outcomes. By concentrating on experiments with clear results, this dataset offers a robust benchmark for comparing the efficiency and effectiveness of reproducibility tools.

Additionally, we resolved all issues related to incomplete dependency specifications, insufficient PL description, and incomplete execution steps across the curated dataset.

Thus, the curated dataset consists of 18 original experiment repositories that were successfully reproduced using at least one of the evaluated reproducibility tools. This dataset is available in [**?**]. Additionally, this dataset serves as a reference for researchers to explore reproducibility challenges and develop improved methodologies.

As detailed in Section 4, the curated dataset retains the diversity of PL and domains while excluding experiments that failed reproducibility assessments. It includes experiments implemented in Python, R, C++, and other languages, covering a wide range of domains. Details of the curated dataset are presented in Table 4. Each experiment of the curated dataset is accompanied by a clear link to its repository, the associated publication, and metadata such as the PL and database engines used and the classification following the *Artifact Review and Badging (v1.1)*

We also analyzed the presence of ACM artifact badges among the original experiments to explore their relationship with actual reproducibility. Initially, 9 experiments appeared to qualify for the Artifacts Available badge. However, upon closer inspection, we had already excluded the two experiments that were implemented as VS Code plugins. For the remaining cases, we removed the badge designation where the associated code was only hosted on GitHub without referencing a specific tag, release, or commit hash — which does not meet the permanence and citability criteria defined by the ACM Artifact Review and Badging guidelines.

6 THREATS TO VALIDITY

In this section, we discuss several potential threats to the validity of our study, which involves the collection and evaluation of computational experiments. Following established methodology guidelines [17, 80], we categorize these threats into four main areas: construct validity, internal validity, external validity, and reliability. Each is discussed in detail below.

Construct Validity. A potential threat is the selection bias in assembling the dataset. While we aimed for diversity by including experiments across multiple disciplines and PLs, the chosen experiments may not fully represent all computational research scenarios. Additionally, our reliance on publicly available repositories may introduce bias. Such repositories are often curated by authors who prioritize certain features or domains. We addressed this threat by using systematic criteria to select experiments from diverse domains, ensuring the diversity of scientific fields and programming environments.

Table 3: Reproducibility of each experiment on each platform and the package size in MB (in parentheses), when applicable

Publication	Binder	Code Ocean	FLINC	PTU	RenkuLab	ReproZip	SciUnit	Whole Tale
Adnan et al. [1]	✗[b]	✓ (3900)	✗[b]	✓ (986.2)	✓	✓ (974.4)	✓ (269.5)	✗[b]
Bai and Zhao [8]	✗[a]	✓ (241.4)	✗[b]	✓ (2100)	✓	✓ (297)	✓ (2000)	✗[b]
Chauhan et al. [13]	✗[b]	✗[c]	✗[b]	✓ (119.6)	✗[b]	✓ (26.4)	✓ (61.5)	✗[b]
Sun et al. [76]	✗[d]	✗[d]	✗[b]	✓ (71.6)	✗[d]	✓ (69.1)	✓ (69.2)	✗[b]
Zhou et al. [85]	✗[b]	✓ (86.2)	✗[b]	✓ (31.0)	✗[b]	✓ (30.6)	✓ (6.9)	✗[b]
Chen et al. [14]	✓	✓ (2.3)	✗[b]	✓ (152.4)	✗[e]	✓ (150)	✓ (148.5)	✗[b]
Kukucka et al. [46]	✗[b]	✗[a]	✗[b]	✓ (1.1)	✗[b]	✓ (225.6)	✓ (1.1)	✗[b]
Nguyen and Grunske [54]	✗[b]	✗[a]	✗[b]	✗[a]	✗[b]	✗[a]	✗[a]	✗[b]
Xie et al. [81]	✓	✓ (10.3)	✗[b]	✓ (2100)	✓	✓ (2100)	✓ (1900)	✗[b]
Hemes [39]	No Data							
Jehn [44]	✓	✓ (2.7)	✗[b]	✓ (75.2)	✓	✓ (73.7)	✓ (74.9)	✗[b]
Lin et al. [48]	✓	✓ (128.8)	✓ (21.2)	✓ (685.4)	✓	✓ (650.1)	✓ (88.9)	✗[b]
Fries [31]	No Data							
Han [38]	No Data							
Kailas et al. [45]	✗[e]	✗[e]	✗[b]	✗[a]	✗[e]	✗[a]	✗[a]	✗[b]
Yang et al. [82]	✗[a]	✗[a]	✗[b]	✗[a]	✗[a]	✗[a]	✗[a]	✗[b]
Agresta et al. [2]	✓	✓ (0.358)	✗[b]	✓ (77.1)	✓	✓ (83.4)	✓ (86.6)	✗[b]
Coupette et al. [19]	No Data							
Doe [24]	✗[a]	✗[a]	✗[b]	✗[a]	✗[a]	✗[a]	✗[a]	✗[b]
Collins et al. [16]	✗[a]	✓ (147.9)	✗[b]	✓ (33.1)	✓	✓ (39.4)	✓ (177.7)	✓
Essawy [29]	Miss Files							
Hai [37]	✓	✗[e]	✗[b]	✓ (6)	✗[e]	✓ (5.7)	✓ (5.7)	✗[b]
Dritsa et al. [25]	✓	✗[a]	✓ (21.2)	✗[b]	✓	✗[b]	✗[b]	-
Cai and Smeeding [12]	✗[a]	Partial (4.6)	✗[b]	Partial (65.5)	Partial	Partial (67.5)	Partial (71.3)	-
Arcidiacono et al. [6]	✗[a]	✗[a]	✗[b]	✗[a]	✗[a]	✗[a]	✗[a]	-
Spinellis et al. [69]	Partial	✗[e]	✗[b]	Partial (0.289)	✗[e]	Partial (0.184)	Partial (0.284)	-
Almås et al. [5]	✗[a]	✗[b]	✗[b]	Partial (97.4)	Partial	Partial (99.6)	✗[a]	-
Ahmed et al. [4]	✗[a]	✗[a]	✗[b]	✗[a]	✗[a]	✗[a]	✗[a]	-
Fronchetti et al. [32]	✓	✓ (135.5)	✗[b]	✓ (99.7)	✓	✓ (10.8)	✓ (99.1)	-

[a] Code crashed
[b] PL not supported
[c] g++ compiler not supported
[d] Database not supported
[e] Python version not supported
[f] Tool/IDE not supported

Internal Validity. There are concerns about the integrity of the experiments collected and potential errors in assembling the dataset. Threats include the accuracy of the computational environment, completeness of dependency information, and correctness of execution instructions provided with the experiments. Each experiment in the curated dataset was thoroughly reviewed to ensure accurate and complete information. However, inconsistencies or omissions in the original repositories might persist. To mitigate this, we resolved all the issues, such as incomplete dependency descriptions and incomplete PL descriptions.

We also acknowledge that dependencies can extend beyond build-level concerns to include OS-level and hardware/architecture-specific requirements (e.g., kernel modules, specific CPU instruction sets, or GPU drivers). These types of dependencies were not addressed in this study, as our focus was on software- and environment-level reproducibility, which represents the most common and widely accessible class of computational experiments. Supporting lower-level or hardware-bound reproducibility remains an important direction for future work.

Table 4: Curated dataset

ID/ Repo. Link	Pub.	PL/Database	Reproduced
E1	[1]	Unix Shell, Python	
E2	[8]	Python	
E3	[13]	C++	
E4	[76]	Python /PostgreSQL	
E5	[85]	C++	
E6	[14]	Python	
E7	[46]	Java, Python, Unix Shell	
E8	[81]	Python	
E9	[44]	Python	
E10	[48]	Python	
E11	[2]	Python	
E12	[16]	R	
E13	[37]	Python /SQLite	
E14	[25]	Jupyter Notebook	
E15	[12]	R	
E16	[69]	R/Python/Unix Shell	
E17	[5]	R	
E18	[32]	Python	

External Validity. Although the curated dataset spans multiple disciplines and computational setups, the experiments primarily focus on academic and open-source experiments. This focus may limit the applicability of the findings to proprietary or industry-driven research environments, which may involve different computational workflows. We acknowledge this limitation and propose this curated dataset as a starting point for evaluating reproducibility tools in academic settings. Future work could extend the dataset to include more industry-oriented experiments.

Reliability. Variations in how experiments are executed across different computational environments (e.g., different Operating System (OS), hardware configurations, or software versions) can pose challenges to reproducibility. These differences may impact the results when using the dataset to assess the performance and robustness of reproducibility tools. To mitigate this threat, we standardized the preparation and documentation process for the experiments on the curated dataset. Local tools were executed within a single, consistent computational environment. For experiments involving online tools, we established new environments for each test to closely simulate the intended computational environments, we ensured that the setup was thoroughly documented, including details on software dependencies, configurations, and execution procedures. Where feasible, we created reproducibility packages using multiple tools to facilitate future reproducibility efforts. These packages, along with their information, are publicly available in the Zenodo repository to serve as a consistent reference [**?**].

7 DISCUSSION

The development of this curated dataset highlights the critical role of standardized benchmarks in evaluating reproducibility tools. Our findings emphasize the challenges that computational researchers face, particularly in ensuring that experiments are well-documented, executable across diverse environments, and supported by existing reproducibility tools.

One of the most striking observations is the significant gap between the number of experiments collected and those successfully reproduced. Of the 38 experiments gathered, only 18 (47%) could be executed using at least one reproducibility tool. The primary barriers included missing data or files (16%) and software version incompatibilities (10%). In several cases, experiments depended on outdated or deprecated packages, or required specific versions of tools (e.g., Python 3.5 or legacy libraries) that conflicted with current environments. Although technologies like containers and virtual environments can help address such issues, their effectiveness is limited when the original documentation is incomplete or when essential configuration files are missing. This reinforces the necessity for standardized and complete documentation, including explicit dependency lists, environment specifications, and detailed execution instructions. Making such information available—such as the reproducible packages we provide and all the description of the experiment—can significantly reduce incompatibility issues and improve the long-term reproducibility of computational experiments.

Furthermore, the dataset reveals disparities in the effectiveness of existing reproducibility tools. While some tools excel at handling Python-based experiments, they struggle with more complex setups, such as experiments requiring specialized databases, multi-language implementations, or older software dependencies. No single tool was capable of executing all the experiments, indicating that current solutions still have substantial limitations. This finding underscores the need for more adaptable and comprehensive tools capable of supporting the diverse requirements of computational research.

A key takeaway from this study is the importance of thoroughly documenting the computational environment and execution details. Experiments that provided detailed computational information—such as specific software environments, precise dependency versions, and clear execution steps—were significantly more likely to be successfully reproduced. This suggests that future efforts in reproducibility should emphasize not only making data and code available but also ensuring that experiments are accompanied by the computational environment and execution details.

Looking ahead, this dataset serves as a foundation for further advancements in reproducibility research. Future iterations should aim to include a broader spectrum of experiments, incorporating industry-driven computational research alongside academic experiments. Additionally, expanding the dataset to cover more computational environments—such as cloud-based platforms and high-performance computing clusters—could help further assess the adaptability of reproducibility tools.

By addressing these challenges, this curated dataset not only provides a robust benchmark for evaluating reproducibility tools but also contributes to the broader effort of fostering transparency, reliability, and collaboration in computational research.

8 CONCLUSION

This work introduces a meticulously curated dataset designed to assess reproducibility tools in computational research. By analyzing a variety of computational experiments, we have highlighted significant gaps in documentation practices, underscoring the urgent need for improvements to ensure reliability and transparency in scientific research.

Our findings reveal that, despite advancements in reproducibility practices, substantial challenges remain, particularly regarding dependency management and environmental compatibility. With only a 47% success rate in experiment reproducibility, the importance of advancing toward more robust and adaptable solutions is clear.

Furthermore, it is imperative that the scientific community commit to adopting more rigorous and transparent documentation practices. Detailed and accessible documentation is essential to enable researchers to effectively verify and reproduce each other's work, thereby improving integrity and accountability in research.

This work highlights the importance of reproducibility in computational research and serves as a wake-up call for researchers to collaborate in the continuous improvement of scientific practices. This dataset serves as a practical resource for evaluating and improving reproducibility tools. Based on the knowledge derived from this work, future efforts can significantly improve the reliability, transparency, and accessibility of computational research.

ACKNOWLEDGMENTS

This work is financed by National Funds through the Portuguese funding agency, FCT - Fundação para a Ciência e a Tecnologia within project LA/P/0063/2020. DOI 10.54499/LA/P/0063/2020 | https://doi.org/10.54499/LA/P/0063/2020. This work is co-financed by Component 5 - Capitalization and Business Innovation, integrated in the Resilience Dimension of the Recovery and Resilience Plan within the scope of the Recovery and Resilience Mechanism (MRR) of the European Union (EU), framed in the Next Generation EU, for the period 2021 - 2026, within project NewSpacePortugal, with reference 11. The first author was also supported by the doctoral Grant SFRH/BD/1513 66/2021 financed by Portuguese Foundation for Science and Technology.

REFERENCES

[1] Muhammad Adnan, Yassaman Ebrahimzadeh Maboud, Divya Mahajan, and Prashant J. Nair. 2022. Accelerating Recommendation System Training by Leveraging Popular Choices. *VLDB Endow.* 15, 1 (jan 2022), 127–140. https://doi.org/10.14778/3485450.3485462

[2] Antonio Agresta, Chiara Biscarini, Fabio Caraffini, and Valentino Santucci. 2023. An Intelligent Optimised Estimation of the Hydraulic Jump Roller Length. In *Applications of Evolutionary Computation*, João Correia, Stephen Smith, and Raneem Qaddoura (Eds.). Springer Nature Switzerland, Cham, 475–490.

[3] Raza Ahmad, Naga Nithin Manne, and Tanu Malik. 2022. Reproducible Notebook Containers using Application Virtualization. In *2022 IEEE 18th International Conference on e-Science (e-Science)*. IEEE Computer Society, Los Alamitos, CA, USA, 1–10. https://doi.org/10.1109/eScience55777.2022.00015

[4] Shibbir Ahmed, Sayem Mohammad Imtiaz, Samantha Syeda Khairunnesa, Breno Dantas Cruz, and Hridesh Rajan. 2023. Design by Contract for Deep Learning APIs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 94–106. https://doi.org/10.1145/3611643.3616247

[5] Ingvild Almås, Alexander W. Cappelen, Erik Ø. Sørensen, and Bertil Tungodden. 2022. Global evidence on the selfish rich inequality hypothesis. *Proceedings of the National Academy of Sciences* 119, 3 (2022), e2109690119. https://doi.org/10.1073/pnas.2109690119 arXiv:https://www.pnas.org/doi/pdf/10.1073/pnas.2109690119

[6] Peter Arcidiacono, Josh Kinsler, and Tyler Ransom. 2022. Legacy and Athlete Preferences at Harvard. *Journal of Labor Economics* 40, 1 (2022), 133–156. https://doi.org/10.1086/713744 arXiv:https://doi.org/10.1086/713744

[7] Nabiha Asghar. 2016. Yelp Dataset Challenge: Review Rating Prediction. arXiv:1605.05362 [cs.CL] https://arxiv.org/abs/1605.05362

[8] Jiyang Bai and Peixiang Zhao. 2022. TaGSim: Type-Aware Graph Similarity Learning and Computation. *Proc. VLDB Endow.* 15, 2 (feb 2022), 335–347. https://doi.org/10.14778/3489496.3489513

[9] Adam Brinckman, Kyle Chard, Niall Gaffney, Mihael Hategan, Matthew Jones, Kacper Kowalik, Sivakumar Kulasekaran, Bertram Ludäscher, Bryce Mecum, Jarek Nabrzyski, Victoria Stodden, Ian J. Taylor, Matthew J. Turk, and Kandace Turner. 2019. Computing environments for reproducibility: Capturing the "Whole Tale". *Future Generation Computer Systems* 94 (2019), 854–867. https://doi.org/10.1016/j.future.2017.12.029

[10] Chris Brunsdon and Alexis Comber. 2021. Opening practice: supporting reproducibility and critical spatial data science. *Journal of Geographical Systems* 23, 4 (oct 2021), 477–496. https://doi.org/10.1007/s10109-020-00334-2

[11] Matthias Bussonnier, Jessica Forde, Jeremy Freeman, Brian Granger, Tim Head, Chris Holdgraf, Kyle Kelley, Gladys Nalvarte, Andrew Osheroff, M Pacer, Yuvi Panda, Fernando Perez, Benjamin Ragan Kelley, and Carol Willing. 2018. Binder 2.0: Reproducible, interactive, sharable environments for science at scale. In *Proc. of the 17th Python in Science Conference*. SciPy Community, WA, USA, 113–120. https://doi.org/10.25080/Majora-4af1f417-011

[12] Yixia Cai and Timothy Smeeding. 2020. Deep and Extreme Child Poverty in Rich and Poor Nations: Lessons from Atkinson for the Fight Against Child Poverty. *Italian Economic Journal* 6, 1 (01 Mar 2020), 109–128. https://doi.org/10.1007/s40797-019-00116-w

[13] Komal Chauhan, Kartik Jain, Sayan Ranu, Srikanta Bedathur, and Amitabha Bagchi. 2021. Answering regular path queries through exemplars. *Proc. VLDB Endow.* 15, 2 (Oct. 2021), 299–311. https://doi.org/10.14778/3489496.3489510

[14] Zhuangbin Chen, Jinyang Liu, Yuxin Su, Hongyu Zhang, Xiao Ling, Yongqiang Yang, and Michael R. Lyu. 2022. Adaptive Performance Anomaly Detection for Online Service Systems via Pattern Sketching. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania). ACM, New York,USA, 61–72. https://doi.org/10.1145/3510003.3510085

[15] Code Ocean. 2016. Code Ocean. https://codeocean.com/, Online, accessed November-2024.

[16] Stephen Collins, Gavin Smart, Ben Albright, and David Crippin. 2016. Food Inspection Evaluation Predictions - Source Code. https://github.com/Chicago/food-inspections-evaluation [Online; accessed February-2024].

[17] Thomas D Cook and D T Campbell. 1979. *Quasi-Experimentation: Design and Analysis Issues for Field Settings*. Houghton Mifflin, Boston.

[18] Lázaro Costa, Susana Barbosa, and Jácome Cunha. 2024. Evaluating Tools for Enhancing Reproducibility in Computational Scientific Experiments. In *Proceedings of the 2nd ACM Conference on Reproducibility and Replicability* (Rennes, France) *(ACM REP '24)*. Association for Computing Machinery, New York, NY, USA, 46–51. https://doi.org/10.1145/3641525.3663623

[19] Corinna Coupette, Dirk Hartung, Janis Beckedorf, Maximilian Böther, and Daniel Martin Katz. 2022. Law Smells (Code). https://doi.org/10.5281/zenodo.6468193 DOI: 10.5281/zenodo.6468193, Software.

[20] Terence Critchlow. 2013. *Data-Intensive Science* (1 st ed.). Chapman & Hall/CRC.

[21] Matthew C. Davis, Sangheon Choi, Sam Estep, Brad A. Myers, and Joshua Sunshine. 2023. NaNofuzz: A Usable Tool for Automatic Test Generation. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1114–1126. https://doi.org/10.1145/3611643.3616327

[22] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine* 29, 6 (2012), 141–142.

[23] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. 2017. Nextflow enables reproducible computational workflows. *Nature Biotechnology* 35, 4 (01 Apr 2017), 316–319. https://doi.org/10.1038/nbt.3820

[24] John Doe. 2023. *AXOM: Combination of Weak Learners, eXplanations to Improve Robustness of Ensemble's eXplanations*. Master's thesis. Politecnico di Milano. https://www.politesi.polimi.it/retrieve/72953cc8-1125-4c72-ab43-d60248ddb337/AXOM__Combination_of_Weak_Learners__eXplanations_to_Improve_Robustness_of_Ensemble_s_eXplanations.pdf

[25] Konstantina Dritsa, Thodoris Sotiropoulos, Haris Skarpetis, and Panos Louridas. 2020. Search Engine Similarity Analysis: A Combined Content and Rankings Approach. In *Web Information Systems Engineering – WISE 2020: 21st International Conference, Amsterdam, The Netherlands, October 20–24, 2020, Proceedings, Part II* (Amsterdam, The Netherlands). Springer-Verlag, Berlin, Heidelberg, 21–37. https://doi.org/10.1007/978-3-030-62008-0_2

[26] Peter D. Dueben, Martin G. Schultz, Matthew Chantry, David John Gagne, David Matthew Hall, and Amy McGovern. 2022. Challenges and Benchmark Datasets for Machine Learning in the Atmospheric Sciences: Definition, Status, and Outlook. *Artificial Intelligence for the Earth Systems* 1, 3 (2022), e210002. https://doi.org/10.1175/AIES-D-21-0002.1

[27] Brandon PM Edwards. 2022. napops: Point Count Offsets for Population Sizes of North American Landbirds. https://doi.org/10.5281/zenodo.5967578 DOI: 10.5281/zenodo.5967578, Software.

[28] Adam Erickson. 2019. Deepland Alpha Release. https://doi.org/10.5281/zenodo.3568468 DOI: 10.5281/zenodo.3568468, Software.

[29] Bakinam Essawy. 2023. VIC_Pre-Processing_Rules. https://github.com/uva-hydroinformatics/VIC_Pre-Processing_Rules.

[30] European Organization For Nuclear Research and OpenAIRE. 2013. Zenodo. https://doi.org/10.25495/7GXK-RD71

[31] Jason Alan Fries. 2021. som-shahlab/trove: Manuscript pre-release code. https://doi.org/10.5281/zenodo.4497214 DOI: 10.5281/zenodo.4497214, Software.

[32] Felipe Fronchetti, David C. Shepherd, Igor Wiese, Christoph Treude, Marco Aurélio Gerosa, and Igor Steinmacher. 2023. Do CONTRIBUTING Files Provide Information about OSS Newcomers' Onboarding Barriers?. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 16–28. https://doi.org/10.1145/3611643.3616288

[33] Mohammad Ganji, Saba Alimadadi, and Frank Tip. 2023. Code Coverage Criteria for Asynchronous Programs. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 1307–1319. https://doi.org/10.1145/3611643.3616292

[34] David Gavaghan. 2018. Problems with the Current Approach to the Dissemination of Computational Science Research and Its Implications for Research Integrity. *Bulletin of Mathematical Biology* 80, 12 (01 Dec 2018), 3088–3094. https://doi.org/10.1007/s11538-018-0499-y

[35] Odd Erik Gundersen. 2021. The fundamental principles of reproducibility. *Philosophical Transactions of the Royal Society A* 379, 2197 (2021), 20200210.

[36] Philip J. Guo and Dawson Engler. 2011. CDE: Using System Call Interposition to Automatically Create Portable Software Packages. In *2011 USENIX Annual Technical Conference (USENIX ATC 11)*. USENIX Association, Portland, OR, 21–21. https://www.usenix.org/conference/usenixatc11/cde-using-system-call-interposition-automatically-create-portable-software

[37] Ton Hai. 2019. Incremental Query Execution. https://bitbucket.org/TonHai/iqe/src/master/ [Online; accessed February-2024].

[38] Tianyu Han. 2021. peterhan91/Medical-Robust-Training: Adversarial Training on Medical Data. https://doi.org/10.5281/zenodo.4926118 DOI: 10.5281/zenodo.4926118, Software.

[39] Kyle Hemes. 2023. Data: The magnitude and pace of photosynthetic recovery after wildfire in California ecosystems. https://doi.org/10.5281/zenodo.7651416 DOI: 10.5281/zenodo.7651416, Software.

[40] Charles Tapley Hoyt. 2018. mesh v0.2.0. https://doi.org/10.5281/zenodo.1488650 DOI: 10.5281/zenodo.1488650, Software.

[41] Peter Ivie and Douglas Thain. 2016. PRUNE: A preserving run environment for reproducible scientific computing. In *2016 IEEE 12th International Conference on e-Science (e-Science)*. IEEE, Los Alamitos, CA, USA, 61–70. https://doi.org/10.1109/eScience.2016.7870886

[42] Peter Ivie and Douglas Thain. 2018. Reproducibility in Scientific Computing. *ACM Comput. Surv.* 51, 3, Article 63 (jul 2018), 36 pages. https://doi.org/10.1145/3186266

[43] Yves Janin, Cédric Vincent, and Rémi Duraffort. 2014. CARE, the Comprehensive Archiver for Reproducible Execution. In *Proceedings of the 1st ACM SIGPLAN Workshop on Reproducible Research Methodologies and New Publication Models in Computer Engineering*. Association for Computing Machinery, New York, NY, USA, Article 1, 7 pages. https://doi.org/10.1145/2618137.2618138

[44] Florian Ulrich Jehn. 2021. zutn/Extreme-Climate-Change: Publication ready. https://doi.org/10.5281/zenodo.4588383 DOI: 10.5281/zenodo.4588383, Software.

[45] Prajwal Kailas, Shinichi Goto, Max Homilius, Calum A MacRae, and Rahul C Deo. 2022. ehr_deidentification. https://doi.org/10.5281/zenodo.6617957 DOI: 10.5281/zenodo.6617957, Software.

[46] James Kukucka, Luís Pina, Paul Ammann, and Jonathan Bell. 2022. CONFETTI: Amplifying Concolic Guidance for Fuzzers. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. IEEE, Pittsburgh, PA, USA, 438–450. https://doi.org/10.1145/3510003.3510628

[47] Jeremy Leipzig, Daniel Nüst, Charles Tapley Hoyt, Karthik Ram, and Jane Greenberg. 2021. The role of metadata in reproducible computational research. , 100322 pages. https://doi.org/10.1016/j.patter.2021.100322

[48] Shuqi Lin, Donald C. Pierson, Robert Ladwig, Benjamin M. Kraemer, and Fenjuan R. S. Hu. 2024. Multi-Model Machine Learning Approach Accurately Predicts Lake Dissolved Oxygen With Multiple Environmental Inputs. *Earth and Space Science* 11, 7 (2024), e2023EA003473. https://doi.org/10.1029/2023EA003473 arXiv:https://agupubs.onlinelibrary.wiley.com/doi/pdf/10.1029/2023EA003473 e2023EA003473 2023EA003473.

[49] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár, and C Lawrence Zitnick. 2014. Microsoft coco: Common objects in context. In *Computer vision–ECCV 2014: 13th European conference, zurich, Switzerland, September 6-12, 2014, proceedings, part v 13*. Springer, Cham, 740–755.

[50] Tanu Malik, Quan Pham, Ian T Foster, F Leisch, and RD Peng. 2014. SOLE: towards descriptive and interactive publications. *Implementing reproducible research* 33 (2014).

[51] Mitch Marcus, Beatrice Santorini, and Mary Ann Marcinkiewicz. 1993. Building a large annotated corpus of English: The Penn Treebank. *Computational linguistics* 19, 2 (1993), 313–330.

[52] Haiyan Meng and Douglas Thain. 2015. Umbrella: A Portable Environment Creator for Reproducible Computing on Clusters, Clouds, and Grids. In *Proceedings of the 8th International Workshop on Virtualization Technologies in Distributed Computing* (Portland, Oregon, USA) *(VTDC '15)*. Association for Computing Machinery, New York, NY, USA, 23–30. https://doi.org/10.1145/2755979.2755982

[53] National Academies of Sciences, Engineering, and Medicine. 2019. *Reproducibility and Replicability in Science*. The National Academies Press, Washington, DC. https://doi.org/10.17226/25303

[54] Hoang Lam Nguyen and Lars Grunske. 2022. BEDIVFUZZ: Integrating Behavioral Diversity into Generator-based Fuzzing. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. IEEE, Pittsburgh, PA, USA, 249–261. https://doi.org/10.1145/3510003.3510182

[55] Daniel Nüst and Edzer Pebesma. 2021. Practical Reproducibility in Geography and Geosciences. *Annals of the American Association of Geographers* 111, 5 (2021), 1300–1310. https://doi.org/10.1080/24694452.2020.1806028

[56] Thomas Pasquier, Matthew Lau, Xueyuan Han, Elizabeth Fong, Barbara S. Lerner, Emery R. Boose, Mercè Crosas, Aaron M. Ellison, and Margo Seltzer. 2018. Sharing and Preserving Computational Analyses for Posterity with encapsulator. *Computing in Science & Engineering* 20, 4 (2018), 111–124. https://doi.org/10.1109/MCSE.2018.042781334

[57] Roger D Peng. 2011. Reproducible research in computational science. *Science* 334, 6060 (2011), 1226–1227. https://doi.org/10.1126/science.1213847

[58] Quan Pham, Tanu Malik, and Ian Foster. 2013. Using provenance for repeatability. In *Proceedings of the 5th USENIX Conference on Theory and Practice of Provenance* (Lombard, IL) *(TaPP'13)*. USENIX Association, USA, 2.

[59] Quan Pham, Tanu Malik, and Ian Foster. 2015. Auditing and Maintaining Provenance in Software Packages. In *Provenance and Annotation of Data and Processes*. Springer, Cham, 97–109. https://doi.org/10.1007/978-3-319-16462-5_8

[60] Stephen R Piccolo and Michael B Frampton. 2016. Tools and techniques for computational reproducibility. *GigaScience* 5, 1 (07 2016), s13742–016–0135–4. https://doi.org/10.1186/s13742-016-0135-4 arXiv:https://academic.oup.com/gigascience/article-pdf/5/1/s13742-016-0135-4/25513324/13742_2016_article_135.pdf

[61] Russell A. Poldrack and Jean-Baptiste Poline. 2015. The publication and reproducibility challenges of shared data. *Trends in Cognitive Sciences* 19, 2 (2015), 59–61. https://doi.org/10.1016/j.tics.2014.11.008

[62] Fabi Prezja. 2023. Deep Fast Vision: Accelerated Deep Transfer Learning Vision Prototyping and Beyond. https://doi.org/10.5281/zenodo.7867100 DOI: 10.5281/zenodo.7867100, Software.

[63] Saïd Qasmi and Aurélien Ribes. 2021. Kriging for Climate Change Package. https://doi.org/10.5281/zenodo.5233947 DOI: 10.5281/zenodo.5233947, Software.

[64] ReproZip. 2016. Making Jupyter Notebooks Reproducible with ReproZip. https://docs.reprozip.org/en/1.x/jupyter.html [Online; accessed April-2024].

[65] Saima Ritonummi, Valtteri Siitonen, Markus Salo, Henri Pirkkalainen, and Anu Sivunen. 2023. Flow Experience in Software Engineering. In *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering* (San Francisco, CA, USA) *(ESEC/FSE 2023)*. Association for Computing Machinery, New York, NY, USA, 618–630. https://doi.org/10.1145/3611643.3616263

[66] Royi Ronen, Marian Radu, Corina Feuerstein, Elad Yom-Tov, and Mansour Ahmadi. 2018. Microsoft Malware Classification Challenge. arXiv:1802.10135 [cs.CR] https://arxiv.org/abs/1802.10135

[67] Rok Roškar, Chandrasekhar Ramakrishnan, Michele Volpi, Fernando Perez-Cruz, Lilian Gasser, Firat Ozdemir, Patrick Paitz, Mohammad Alisafaee, Philipp Fischer, Ralf Grubenmann, Eliza Harris, Tasko Olevski, Carl Remlinger, Luis Salamanca, Elisabet Capon Garcia, Lorenzo Cavazzi, Jakub Chrobasik, Darlin Cordoba Osnas, Alessandro Degano, Jimena Dupre, Wesley Johnson, Eike Kettner, Laura Kinkead, Sean D. Murphy, Flora Thiebaut, and Olivier Verscheure. 2023. Renku: a platform for sustainable data science. In *Advances*

in Neural Information Processing Systems, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine (Eds.), Vol. 36. Curran Associates, Inc., NY, USA, 42161–42173. https://proceedings.neurips.cc/paper_files/paper/2023/file/838694e9ab6b0a193b84daaafcac0eed-Paper-Datasets_and_Benchmarks.pdf

[68] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. 2015. Imagenet large scale visual recognition challenge. *International journal of computer vision* 115 (2015), 211–252.

[69] Diomidis Spinellis, Panos Louridas, and Maria Kechagia. 2016. The evolution of C programming practices: a study of the Unix operating system 1973–2015. In *Proceedings of the 38th International Conference on Software Engineering* (Austin, Texas) *(ICSE '16)*. Association for Computing Machinery, New York, NY, USA, 748–759. https://doi.org/10.1145/2884781.2884799

[70] Vicky Steeves, Rémi Rampin, and Fernando Chirigati. 2020. Reproducibility, preservation, and access to research with ReproZip and ReproServer. *IASSIST Quarterly* 44, 1-2 (Jun. 2020), 1–11. https://doi.org/10.29173/iq969

[71] Victoria Stodden. 2010. The Scientific Method in Practice: Reproducibility in the Computational Sciences. *MIT Sloan Research* Paper No. 4773-10 (February 9 2010). https://doi.org/10.2139/ssrn.1550193

[72] Victoria Stodden, Marcia McNutt, David H. Bailey, Ewa Deelman, Yolanda Gil, Brooks Hanson, Michael A. Heroux, John P.A. Ioannidis, and Michela Taufer. 2016. Enhancing reproducibility for computational methods. *Science* 354, 6317 (2016), 1240–1241. https://doi.org/10.1126/science.aah6168 arXiv:https://www.science.org/doi/pdf/10.1126/science.aah6168

[73] Victoria Stodden and Sheila Miguez. 2014. Best practices for computational science: Software infrastructure and environments for reproducible and extensible research. *Journal of open research software* 2, 1 (2014), e21–e21. https://doi.org/10.5334/jors.ay

[74] Victoria Stodden, Sheila Miguez, and Jennifer Seiler. 2015. ResearchCompendia.org: Cyberinfrastructure for Reproducibility and Collaboration in Computational Science. *Computing in Science and Engg.* 17, 1 (jan 2015), 12–19. https://doi.org/10.1109/MCSE.2015.18

[75] Victoria Stodden, Jennifer Seiler, and Zhaokun Ma. 2018. An empirical analysis of journal policy effectiveness for computational reproducibility. *Proc. Natl. Acad. Sci. U. S. A.* 115, 11 (March 2018), 2584–2589.

[76] Ji Sun, Jintao Zhang, Zhaoyan Sun, Guoliang Li, and Nan Tang. 2021. Learned cardinality estimation: a design space exploration and a comparative evaluation. *Proc. VLDB Endow.* 15, 1 (Sept. 2021), 85–97. https://doi.org/10.14778/3485450.3485459

[77] Douglas Thain and Miron Livny. 2005. Parrot: An application environment for data-intensive computing. *Scalable Computing: Practice and Experience* 6, 3 (2005), 9–18.

[78] D. Ton That, G. Fils, Z. Yuan, and T. Malik. 2017. Sciunits: Reusable Research Objects. In *2017 IEEE 13th International Conference on e-Science (e-Science)*. IEEE Computer Society, Los Alamitos, CA, USA, 374–383. https://doi.org/10.1109/eScience.2017.51

[79] Lukas M. Weber, Wouter Saelens, Robrecht Cannoodt, Charlotte Soneson, Alexander Hapfelmeier, Paul P. Gardner, Anne-Laure Boulesteix, Yvan Saeys, and Mark D. Robinson. 2019. Essential guidelines for computational method benchmarking. *Genome Biology* 20, 1 (20 Jun 2019), 125. https://doi.org/10.1186/s13059-019-1738-8

[80] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2012. *Experimentation in software engineering*. Vol. 9783642290442. Springer-Verlag Berlin Heidelberg, Heidelberg, Germany. 1 – 236 pages. https://doi.org/10.1007/978-3-642-29044-2

[81] Huan Xie, Yan Lei, Meng Yan, Yue Yu, Xin Xia, and Xiaoguang Mao. 2022. A Universal Data Augmentation Approach for Fault Localization. In *Proceedings of the 44th International Conference on Software Engineering* (Pittsburgh, Pennsylvania). Association for Computing Machinery, New York, USA, 48–60. https://doi.org/10.1145/3510003.3510136

[82] Feihong Yang, Xuwen Wang, Hetong Ma, and Jiao Li. 2021. Transformers-sklearn: a toolkit for medical language understanding with transformer-based models. *BMC Medical Informatics and Decision Making* 21, 2 (30 Jul 2021), 90. https://doi.org/10.1186/s12911-021-01459-0

[83] Andrew Youngdahl, Dai-Hai Ton-That, and Tanu Malik. 2019. SciInc: A Container Runtime for Incremental Recomputation. In *2019 15th International Conference on eScience (eScience)*. IEEE, San Diego, CA, USA, 291–300. https://doi.org/10.1109/eScience.2019.00040

[84] Chen Zhang, Bihuan Chen, Xin Peng, and Wenyun Zhao. 2022. Buildsheriff: Change-Aware Test Failure Triage for Continuous Integration Builds. In *2022 IEEE/ACM 44th International Conference on Software Engineering (ICSE)*. IEEE, Pittsburgh, PA, USA, 312–324. https://doi.org/10.1145/3510003.3510132

[85] Alexander Zhou, Yue Wang, and Lei Chen. 2022. Butterfly Counting on Uncertain Bipartite Graphs. *Proc. VLDB Endow.* 15, 2 (feb 2022), 211–223. https://doi.org/10.14778/3489496.3489502